Chapter

13

APPLET PROGRAMMING

13.1 Introduction

An applet is a Java program that can be embedded into a web page. It runs inside the web browser and works at client side. An applet is embedded in an HTML page using the APPLET or OBJECT tag and hosted on a web server. There are two types of applets:

- Local Applets
- Remote Applets

Local Applets: An applet developed locally and stored in a local system is known as local applet. Therefore, when webpage is trying to find local applet it does not need the internet connection.

Specifying a Local Applet:

```
<applet codebase="path" code="CS.class" width=120 height=120 >
</apple>
```

Remote Applets: A remote applet is that which is developed by someone else and stored on a remote computer connected to the internet. If our system is connected to the internet, then we can download it from remote computer and run it. In order to locate and load a remote applet, we must know the applet's address on the web. This address is known as Uniform Resource locator(URL) and must be specified in applet's document.

Specifying a Remote Applet:

```
<applet codebase="URL" code="CS.class" width=120 height=120 >
</applet>
```

13.2 How Applets Differ from Applications

There are some important differences between an applet and a standalone Java application, including the following:

- All applets are sub-classes of java.applet.Applet class.
- Applets are not stand-alone programs. Instead, they run within either a web browser or an applet viewer.
- In general, execution of an applet does not begin at main() method.
- Output of an applet window is not performed by System.out.println(). Rather it is handled with various AWT methods, such as drawString().

- Applets are designed to be embedded within an HTML page.
- Applets cannot communicate with other servers on the network.
- The JVM on the user's machine creates an instance of the applet class and invokes various methods during the applet's lifetime.

13.3 Preparing to Write Applets

The following are the steps that are involved in developing and testing an applet:

- Building an applet code(.java file).
- Creating an executable applet(.class file).
- Designing a web page using HTML.
- Preparing <Applet Tag>.
- Incorporating <Applet> tag into the web page.
- Creating HTMl file.
- Testing the applet code.

13.4 Building Applet Code

There are various steps involved in developing and testing an applet. (Ref. Topic 13.3).

To building the applet code two classes of java library are essential namely Applet and Graphics. The Applet class is contained in java.appletpackage provides life and behavior to the applet through its methods such as int(), start(), etc..

The Graphics class contained the method paint() to display the output.This requires that the applet code imports the java.awtpackage that contain the Graphic class.

The general format of applet code is as follows :

```
import java.awt.*;
import java.applet.*;
.....
.....
public class applet classname extends Applet
{
.....
..... // statements
.....
public void paint(Graphics g)
{
.....
..... //Applet operations code
.....
}
```

```
.....
.....
}
```

Example:

```
// display string
import java.awt.*;
import java.applet.*;
public class ExApplet extends Applet
{
 public void paint(Graphics g)
 {
 g.drawString("Computer Science", 50, 100);
 }
}
```

The above program should be saved with the file name **ExApplet.java** in a java subdirectory. Java requires that the main applet class to be declared as public. Therefore, The **public** keyword for the class **ExApplet.**

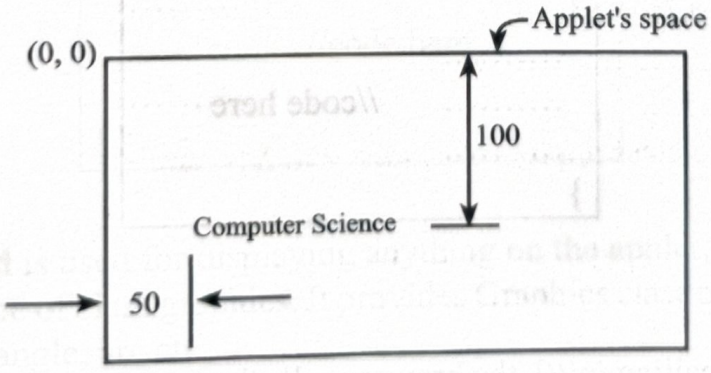
The applet contains only one executable statement,

g.drawString("Computer Science", 50, 100);

When executed, draws the string

Computer Science

at the position 50,100 (pixels) of the applets reserved space as shown in below:



13.5 Applet Life Cycle

Applet life cycle defined as how the object created, started, stopped and destroyed during the entire execution of the application is said to applet life cycle shown in figure 13.1.

- Born state
- Running state
- Idle state
- Dead state

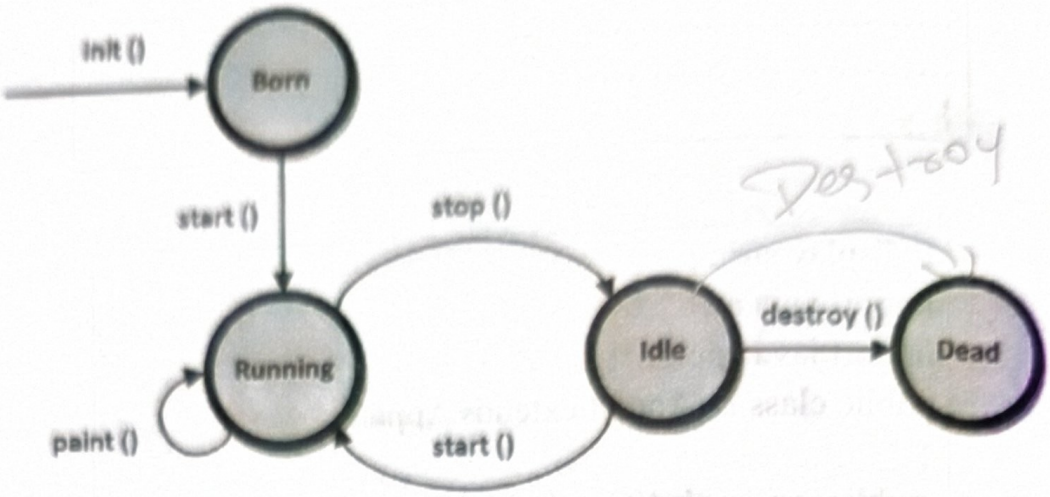


Figure 13.1 An applet state transition diagram

Born State (Initialization State)

Initialization can be done through ***init()***. The init() method is the first method to be called. It is used to initialize the applet each time it is reloaded. Applets can be used for setting up an initial state,

- Creating objects needed by the applet
- Set up initial values
- Loading images or fonts,
- Set up colors
- Setting parameters

Example:

```
public void init()
{
  ..........
  ..........      //code here
  ..........
}
```

Running State

Immediately after calling *init()*, the browser calls the ***start()*** method. *start()* is also called when user returns to an HTML page that contains the applet. So, the user leaves a web page and come back, the applet resumes execution at *start()*. So, when the applet called the *start()* Method it is called its running state.

Example:

```
public void start()
{
  .....
  .....      //code here
  .....
}
```

Idle State (Stopped State)

When we leave the page containing the currently running applet, then it stop running and becomes idle. We can also do so by calling ***stop()*** Method explicitly.

Example:

```
public void stop()
{
  .....
  .....      //code here
  .....
}
```

Dead State

When an applet is completely removed from the Memory, it is called dead. This occurs automatically by invoking the ***destroy()*** method when we quit the browser Like initialization, dead state occurs only once in the applet's life cycle.

Example:

```
public void destroy()
{
  .....
  .....      //code here
  .....
}
```

It is the process where applet get destroyed

Display State

The ***paint()*** method is used for displaying anything on the applet, *paint()* method takes an argument an instance of class graphics. It provides Graphics class object that can be used for drawing oval, rectangle, arc etc.

Example:

```
public void paint(Graphics g)
{
  .....
  .....      //code here
  .....
}
```

13.6 Designing a Web Page

Java applets are programs that reside on web page. A web page is basically made up of text and HTML tags that can be interpreted by a web browser or an applet viewer. A web page is also called HTML page or HTML document. Web pages are stored using a file extension ***.html***. Such files are referred to as HTML files. HTML files should be stored in the same directory as the compiled code of the applets.

A web page is marked by an opening HTML tag <HTML> and closing HTML tag </HTML> and is divided into the following three major parts shown in figure 13.2.

- Comment Section (Optional)
- Head Section (Optional)
- Body Section

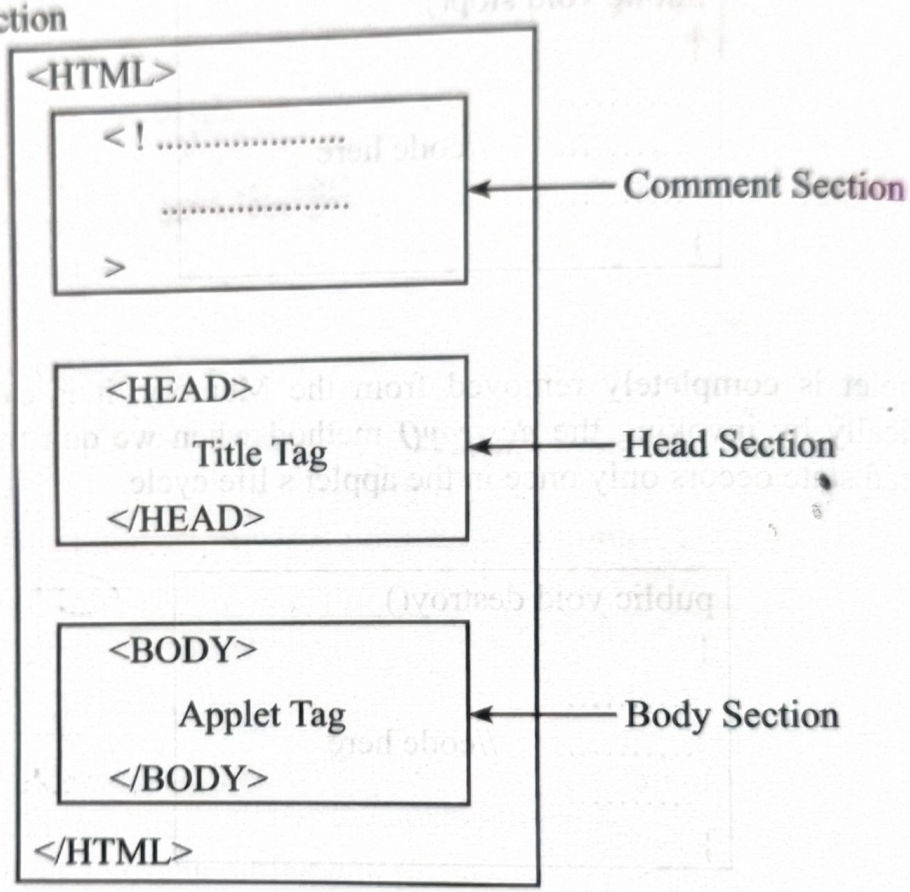


Figure 13.2 A web page templet

Comment Section

A comment line begins with <! and ends with a > and the web browsers will ignore the text enclosed between them. The comments are optional and can be included anywhere in the web page.

Head Section

This section contains title, heading and sub heading of the web page. The head section is defined with a starting <Head> tag and closing </Head> tag.

```
<Head>
<Title> Welcome to Java Applet </Title>
</Head>
```

Body Section

This section contains entire information and behaviour of the web page. It defines what the data placed and where on to the screen. It describes the color, location, sound etc. of the data or information that is going to be placed in the web page.

```
<Body>
<Center>
<H1> Welcome to Java </H1>
</Center>
<BR>
<APPLET>

</APPLET>
</Body>
```

13.7 Applet Tag

The <Applet> tag supplies the name of the applet to be loaded and tells the browser how much space the applet requires. The ellipsis in the tag <Applet> indicates that it contains certain attributes that must specified. The <Applet> tag given below specifies the minimum requirements to place the ***ExApplet.java*** applet on a web page.

```
<Applet code = ExApplet.class width = 400 Height = 400 >

</Applet>
```

This HTML code tells the browser to load the compiled java applet *ExApplet.class*, which is in the same directory as this HTML file. It also specifies the display area for the applet output as 400 pixels width and 400 pixels height.

The applet tag discussed above specified the three things:

- Name of the applet
- Width of the applet (in pixels)
- Height of the applet (in pixels)

We can make this display area appear in the center of screen by using the CENTER tags as shown below:

```
<CENTER>
<Applet>
.....
.....
.....
</Applet>
</CENTER>
```

Attributes of an Applet tag

Required Attributes

Attribute	Value	Description
code	*URL*	Specifies the file name of a Java applet
object	*name*	Specifies a reference to a serialized representation of an applet

Optional Attributes

Attribute	Value	Description
align	left right top bottom middle baseline	Specifies the alignment of an applet according to surrounding elements
alt	*text*	Specifies an alternate text for an applet
archive	*URL*	Specifies the location of an archive file
codebase	*URL*	Specifies a relative base URL for applets specified in the code attribute
height	*pixels*	Specifies the height of an applet
hspace	*pixels*	Defines the horizontal spacing around an applet
name	*name*	Defines the name for an applet (to use in scripts)
vspace	*pixels*	Defines the vertical spacing around an applet
width	*pixels*	Specifies the width of an applet

13.8 Creating an Executable Applet

Executable applet is nothing but the ***.class*** file of applet, which is obtained by compiling the source code of the applet. Compiling the applet is exactly the same as compiling an application using following command.

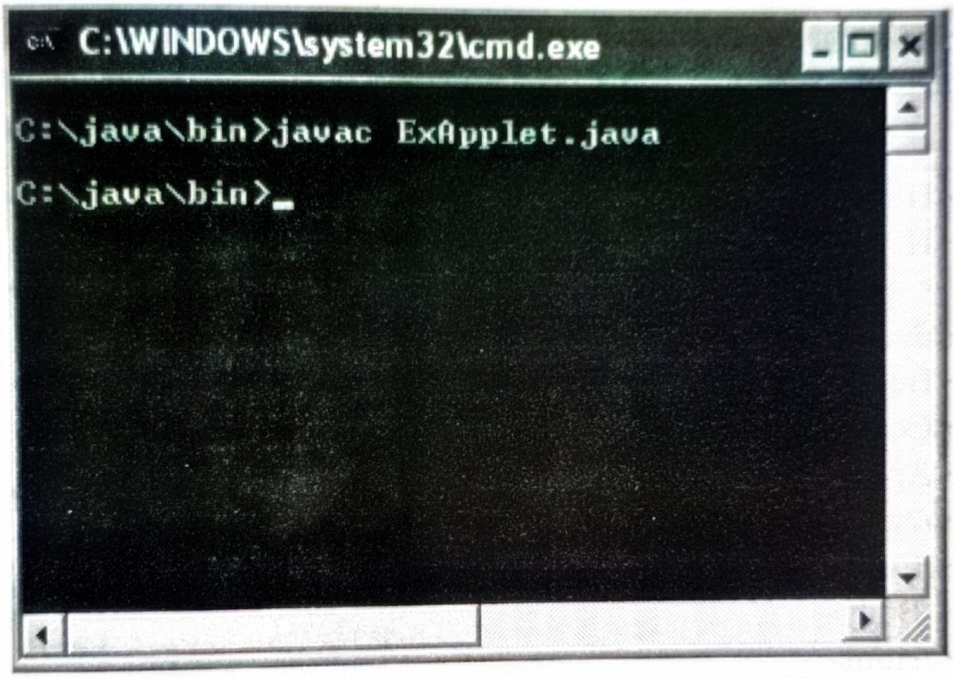
C:\java\bin> ***javac appletname.java***

Let us consider thefollowing program will be saved with named ***ExApplet.java:***

```
import java.awt.*;
import java.applet.*;
public class ExApplet extends Applet
{
 public void paint(Graphics g)
 {
 g.drawString("Computer Science", 50, 100);
 }
}
```

After creating *ExApplet.java* file, we need to compile that file as shown below:

c:\java\bin> javac ExApplet.java



The compiled output file called ***ExApplet.class*** should be placed in the same directory as the source file. If any error message is received, then we must check for errors, correct them and compile the applet again.

13.9 Adding Applet to HTML File

Let us consider the following program will be saved with named ***ExApplet.java.***

```
// display string
import java.awt.*;
import java.applet.*;
public class ExApplet extends Applet
{
 public void paint(Graphics g)
 {
 g.drawString("Computer Science", 50, 100);
 }
}
```

To execute an applet in a web browser, we need to write a short HTML text file that contains the appropriate APPLET tag. The following HTML program executes **ExApplet. java.**

```
<Html>
<Head>
   <Title> Welcome to the Applet </Title>
</Head>
<Body>
<Center>
<Applet code=ExApplet.class  Width=400   Height=400>
</Applet>
</Center>
</Body>
</Html>
```

The *code* attribute specify the *ExApplet.class* file. The *width* and *height* attributes specify the dimensions of the display area used by the applet. After you create this file, you can execute the HTML file called *ExApplet.html* on the command line or using web browser.

13.10 Running the Applet

To run an applet, we require one of the following tools:

- Java-enabled web browser
- Java appletviewer

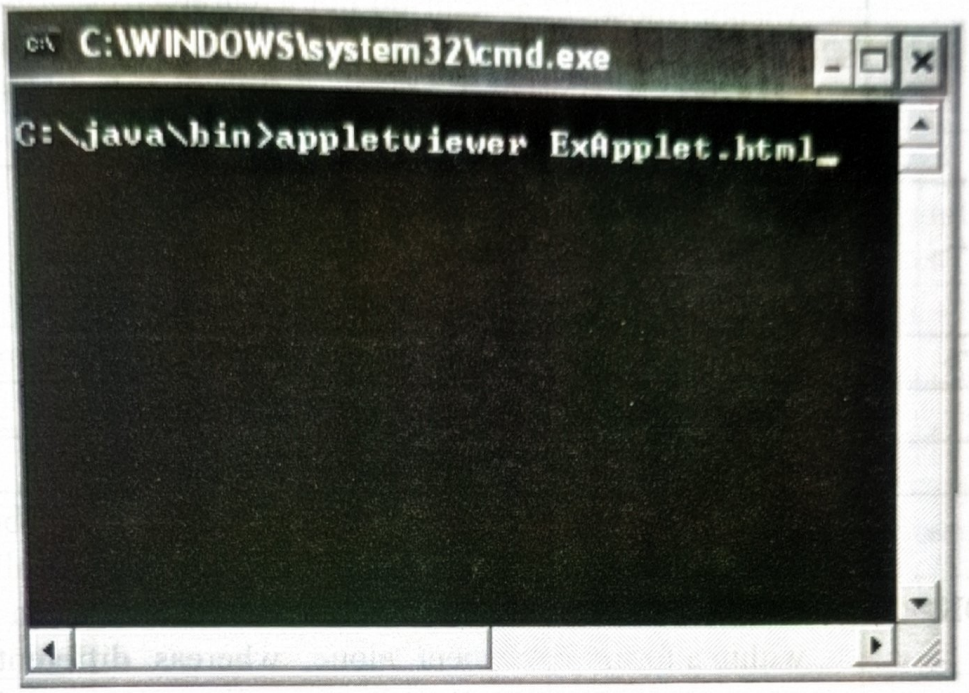
Java-enabled web browser

If we use a *Java-enabled Web browser*, we will be able to see the entire Web page containing the applet. For example, *Google chrome, Firefox, etc.*

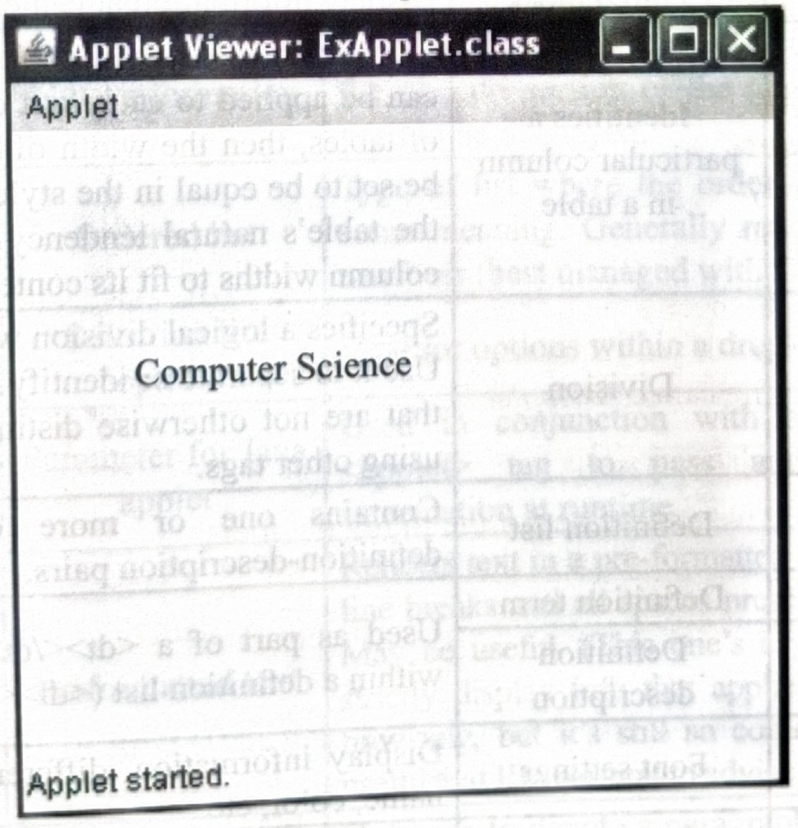
Java appletviewer

If we use the *appletviewer* tool, we will only see the applet output. The appletviewer is available as part of the Java Development Kit that we have been using so far. For example, if we want to run our applet (ExApplete.html) as follows:

C:\java\bin>appletviewer ExApplet.html



Output



13.11 More About HTML Tag

The following table shows the important HTML tag and their functions.

Tag	Description	Usage
<A>	Anchor (most commonly a link)	Use to create links in content. Use the title attribute whenever the contents of the <a> ... pair do not accurately describe what you'll get from selecting the link. Title attribute often displays as a tooltip in visual browsers, which may be a helpful usability aid.
<APPLET>	Inserts a Java applet	<Applet code = ExApplet.class width = 400 Height = 400> </Applet>
<BODY>	Document body	Essential (unless you're using frames)
 	Line break	Line break or inserting new line
<B>	Bold text	For bold text / Formatting text
<BUTTON>	Used for a standard clickable button within a form	Often better than <input type="button" /> or <input type="submit" />, as it allows you to assign different styles based on the HTML element alone, whereas differentiating style based on the type of input is less well supported.
<CENTER>	Centred block	Display information at center of the screen.
<COL>	Identifies a particular column in a table	Can be very useful. e.g. <col class="namecol"> can be applied to each first column in a series of tables, then the width of each column may be set to be equal in the style sheet, overriding the table's natural tendency to adjust its own column widths to fit its contents.
<DIV>	Division	Specifies a logical division within a document. Use it to separate or identify chunks of content that are not otherwise distinguished naturally using other tags.
<DL>	Definition list	Contains one or more definition-term / definition-description pairs.
<DT>	Definition term	Used as part of a <dt></dt><dd></dd> pair within a definition list (<dl></dl>)
<DD>	Definition description	
<FONT>	Font settings	Display information, different font size, font name, color, etc..
<FORM>	Input form	Essential for data input

<H1>	Level 1 header	Aim to have one H1 on each page, containing a description of what the page is about.
<HEAD>	Document head	Essential. Contains information about a page that does not constitute content to be communicated as part of the page.
<HR>	Horizontal rule	Display info with no semantic value – never use it. "Horizontal", by definition, is a visual attribute.
<HTML>		Core element of every web page.
<IMG >	Show an image	Vital. Always use the alt or longdesc attributes when the image has content value
<INPUT>	Input fields within forms	Vital. (I prefer to use <button> for buttons and submit buttons though)
<I>	Italicised text	For italic text / formatting text
<LINK>	Defines a relationship to another document	Commonly used to reference external stylesheets, but has other minor uses
<LI>	List item	Specifies an item in an unordered or ordered list (or)
<MARQUEE>	Makes text scroll across the screen	For Scrolling text
<META>	Meta-information	Useful way to insert relevant information into the <head> section of the page that does not need to be displayed.
<OL>	Ordered list	Type of list where the order of elements has some meaning. Generally rendered with item numbers (best managed with CSS).
<OPTION>	Selection list option	Vital for options within a drop-down control.
<PARAM>	Parameter for Java applet	Used in conjunction with an <object> or <applet> tag to pass additional setting information at runtime.
<PRE>	Preformatted text	Renders text in a pre-formatted style, preserving line breaks and all spaces present in the source. May be useful. (This one's a paradox, as it is strictly display info that applies only to visual browsing, but it's still so commonly used and useful that I'm hesitant to advise against using it.)
<P>	Paragraph	Only use to denote a paragraph of text. Never use for spacing alone.

<SCRIPT>	Inline script (e.g. JavaScript)	It's better to have all scripts as separate files than to write inline or in the <head>section, however still has its uses.
<SELECT>	Selection list	A drop-down selector for a form.
<SPAN>	An inline span within text	Use to apply meaning (and style) to a span of text that goes with the flow of content (whereas a tag is block-level and breaks the flow)
<STRONG>	Strong emphasis	Use this instead of the old <b> tag.
<STYLE>	CSS style settings	Normally used in <head> section of a page. Try to use external style sheets, to enable you to apply different styles for different output media.
<SUB>	Subscript text	Arguably display info – recommend using alternative tags (e.g. <cite>). May be required in some academic uses, e.g. Chemical formulas.
<SUP>	Superscript text	
<TABLE>	Table	Use for repeated data that has a naturally tabular form. Never use for layout purposes.
<TD>	Table data cell	A cell containing actual data. If a cell actually contains a descriptor or identifier for a row or column, use a <th> (table header) tag, not a <td>. This usually applies to column headers (within a <thead>), column footers (within a <tfoot>), as well as row headers (usually the first cell in a row in the <tbody>).
<TEXTAREA>	Multi-line text input area in a form	Essential
<TH>	Table column or row header cell	May appear in a <thead> (to denote a column header cell), <tbody> (to denote a row header), and in <tfoot> (to denote a column foot cell, e.g. a total)
<TITLE>	Document title	Essential
<TR>	Table row	Essential with tables
<TT>	"Teletype" – simulates typewriter output	Similar to <pre>, except that it collapses white space like normal HTML (whereas <pre> leaves all consecutive white space intact). Avoid if possible
<UL>	Unordered list	Essential. Use for lists where the order or items has no particular importance.
<U>	Underline text	For underling text / formatting text

13.12 Display Numerical Values in Applet

In applets, we can display numerical values by first converting them into strings and then using the drawstring() method of ***Graphics*** class. We can do this easily by calling the valueOf() Method of String class.

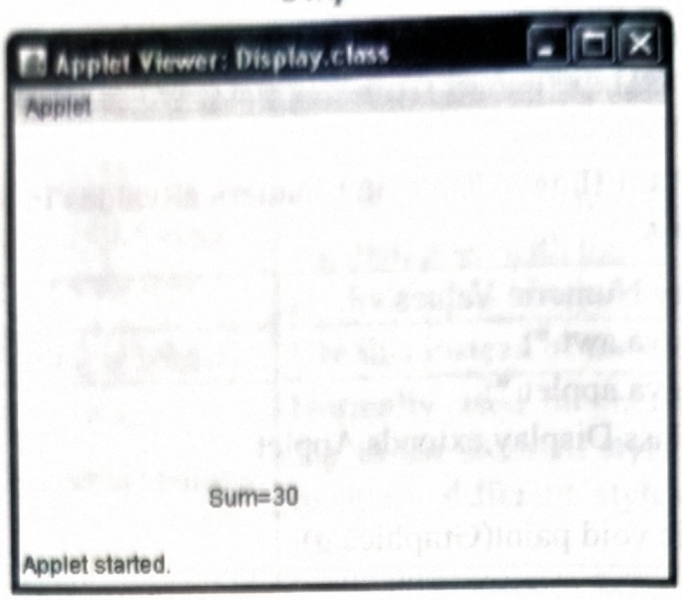
The following program shows displaying numerical values in an applet will be saved with named ***Display.java.***

```
// Display Numeric Values
import java.awt.*;
import java.applet.*;
public class Display extends Applet
{
    public void paint(Graphics g)
    {
          int val1 = 10;
          int val2 = 20;
          int sum = val1 + val2;
         String str_sum = "Sum="+String.valueOf(sum);
         g.drawString(str_sum,100,200);
    }
}
```

The compiled output file called ***Display.class*** should be placed in the same directory as the source file. After creating *Display.class*file, we need create ***Display.html*** file and run that as shown below:

```
<Html>
<Head>
   <Title>Display Numeric Values</Title>
</Head>
<Body>
<Center>
<Applet code=Display.class  Width=400  Height=400>
</Applet>
</Center>
</Body>
</Html>
```

Output



13.13 Getting Input from the User in Java Applet

Applets work in graphical environment. Therefore applets treat input as text strings. We must first create an area of the screen in which user can type and edit input items. We can do this by using the TextField class of the applet package. The values of the fields can be given even editer after the creation of input fields. Next step is to retrieve the items from the fields for display of calculations.

For any kinds of computation on the input field, we must convert it to the right form and the results again converted back to strings for display.

The following program shows getting input from the user in java applet will be saved with named ***DemoUserIn.java.***

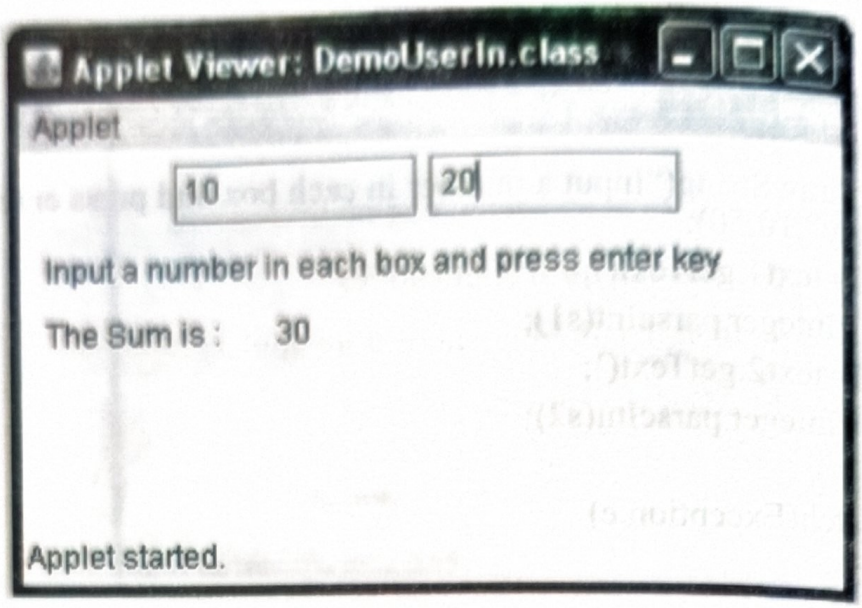
```
//Program to find sum of two integers using applet
importjava.awt.*;
import java.applet.*;
public class DemoUserIn extends Applet
{
TextField text1,text2;
public void init()
 {
text1=new TextField(10);
text2=new TextField(10);
add(text1);
add(text2);
}
public void paint (Graphics g)
{
```

```
int x=0,y=0,z=0;
String s1,s2,s;
try
 {
g.drawString("Input a number in each box and press enter
key",10,50);
s1=text1.getText();
x=Integer.parseInt(s1);
s2=text2.getText();
y=Integer.parseInt(s2);
}
catch(Exception e)
{
}
z=x+y;
s=String.valueOf(z);
g.drawString("The Sum is :",10,75);
g.drawString(s,100,75);
}
public boolean action (Event e, Object ob)
{
repaint();
return (true);
}
}
```

The compiled output file called ***DemoUserIn.class*** should be placed in the same directory as the source file. After creating *DemoUserIn.class*file, we need create ***DemoUserIn.html*** file and run that as shown below:

```
<Html>
<Head>
<Title> Sum of Two Integers </Title>
</Head>
<Body>
<Center>
<Applet code= DemoUserIn.class  Width=400  Height=400>
</Applet>
</Center>
</Body>
</Html>
```

Output



13.14 Advantages and Disadvantages of Applet

Advantages

- As applet is a small java program, so it is platform independent code which is capable to run on any browser.
- Applets can perform various small tasks on client-side machines. They can play sounds, show images, get user inputs, get mouse clicks,etc.
- Applets creates and edit graphics on client-side which are different and independent of client-side platform.
- Applets are secure and safe to use because they cannot perform any modifications over local system.
- Various small tasks such as performing login, inventory checking, task scheduling can be done by applets running over Intranets.

Disadvantages

- Applet itself cannot run or modify any application on the local system.
- Applets has no access to client-side resources such as files, OS etc.
- Applet has little restriction when it comes to communication. It can communicate only with the machine from which it was loaded.
- Applet cannot work with native methods.

Review Questions

1. What is an applet?
2. Differentiate between local and remote applet.
3. Explain how applets differ from applications.
4. Explain steps involved developing and testing applets.
5. Write a note on building an applet.
6. Explain applet life cycle model.
7. Write a note on designing webpage.
8. Write a note on <Applet> tag.
9. Explain creating and executing applet.
10. Write a note on adding applet to html file.
11. Write a note html tags.
12. Explain with example how to display numeric values in an applet.
13. Explain getting input from the user in an applet.
14. List the advantages and disadvantages of an applet.